

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
APPLIED BUSINESS

Practical – VI

Roll No.: T044	Name: SACHIN SINGH
Class: TY.IT	Batch: 1
Date of Assignment: 02/09/2026	Date/Time of Submission: 02/09/2026

#Q1. Load the dataset and display the first 10 records

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_csv("supermarket_sales - supermarket_sales.csv")
# Display first 10 records
print(df.head(10))
```

	invoice_id	branch	city	customer_type	gender_customer
0	750-67-8428	A	Yangon	Member	Female
1	226-31-3081	C	Naypyitaw	Normal	Female
2	631-41-3108	A	Yangon	Normal	Male
3	123-19-1176	A	Yangon	Member	Male
4	373-73-7010	A	Yangon	Normal	Male
5	699-14-3026	C	Naypyitaw	Normal	Male
6	355-53-5943	A	Yangon	Member	Female
7	315-22-5665	C	Naypyitaw	Normal	Female
8	665-32-9167	A	Yangon	Member	Female
9	692-92-5582	B	Mandalay	Member	Female

	product_line	unit_cost	quantity	Spct_markup	revenue
0	Health and beauty	74.69	7	26.1415	548.9715
1	Electronic accessories	15.28	5	3.8200	80.2200
2	Home and lifestyle	46.33	7	16.2155	340.5255
3	Health and beauty	58.22	8	23.2880	489.0480
4	Sports and travel	86.31	7	30.2085	634.3785
5	Electronic accessories	85.39	7	29.8865	627.6165
6	Electronic accessories	68.84	6	20.6520	433.6020
7	Home and lifestyle	73.56	10	36.7800	772.3800
8	Health and beauty	36.26	2	3.6160	76.1460
9	Food and beverages	54.84	3	8.2260	172.7460

	date	time	payment_method	cogs	gm_pct	gross_income	rating
0	01/05/19	13:08	Ewallet	522.83	4.761905	26.1415	9.1
1	03/08/19	10:29	Cash	76.40	4.761905	3.8200	9.6
2	03/03/19	13:23	Credit card	324.31	4.761905	16.2155	7.4
3	1/27/2019	20:33	Ewallet	465.76	4.761905	23.2880	8.4
4	02/08/19	10:37	Ewallet	604.17	4.761905	30.2085	5.3
5	3/25/2019	18:30	Ewallet	597.73	4.761905	29.8865	4.1
6	2/25/2019	14:36	Ewallet	413.04	4.761905	20.6520	5.8
7	2/24/2019	11:38	Ewallet	735.60	4.761905	36.7800	8.0

#Q2. Display number of records, columns, column names and data types

```
print("Number of records:", df.shape[0])
print("Number of columns:", df.shape[1])
print("\nColumn names:")
print(df.columns)
print("\nData types:")
print(df.dtypes)
```

```

Number of records: 1800
Number of columns: 17

Column names:
Index(['invoice_id', 'branch', 'city', 'customer_type', 'gender_customer',
       'product_line', 'unit_cost', 'quantity', 'Spct_markup', 'revenue',
       'date', 'time', 'payment_method', 'cogs', 'gm_pct', 'gross_income',
       'rating'],
      dtype='object')

Data types:
invoice_id      object
branch          object
city            object
customer_type   object
gender_customer object
product_line    object
unit_cost       float64
quantity        int64
Spct_markup     float64
revenue         float64
date            object
time            object
payment_method  object
cogs            float64
gm_pct          float64
gross_income    float64
rating          float64
dtype: object
```

#Q3. Display all unique values of product_line

```
print(df["product_line"].unique())
```

```
... ['Health and beauty' 'Electronic accessories' 'Home and lifestyle'
     'Sports and travel' 'Food and beverages' 'Fashion accessories']
```

#Q4. Calculate total revenue generated by each product line

```
total_revenue = df.groupby("product_line")["revenue"].sum()
```

```
print(total_revenue)
```

```
... product_line
     Electronic accessories    54337.5315
     Fashion accessories      54305.8950
     Food and beverages       56144.8440
     Health and beauty        49193.7390
     Home and lifestyle       53861.9130
     Sports and travel        55122.8265
     Name: revenue, dtype: float64
```

#Q5. Calculate average revenue generated by each product line

```
average_revenue = df.groupby("product_line")["revenue"].mean()
```

```
print(average_revenue)
```

```
... product_line
     Electronic accessories    319.632538
     Fashion accessories      305.089298
     Food and beverages       322.671517
     Health and beauty        323.643020
     Home and lifestyle       336.636956
     Sports and travel        332.065220
     Name: revenue, dtype: float64
```

#Q6. Calculate total quantity sold for each product line

```
total_quantity = df.groupby("product_line")["quantity"].sum()
```

```
print(total_quantity)
```

```
... product_line
     Electronic accessories    971
     Fashion accessories      902
     Food and beverages       952
     Health and beauty        854
     Home and lifestyle       911
     Sports and travel        920
     Name: quantity, dtype: int64
```

#Q7. Calculate the median value of the revenue column

```
median_revenue = df["revenue"].median()
```

```
print("Median Revenue:", median_revenue)
```

```
... Median Revenue: 253.848
```

#Q8. Classify every transaction as High Revenue or Low Revenue

```
df["revenue_level"] = np.where(
    df["revenue"] >= median_revenue,
    "High Revenue",
    "Low Revenue"
```

```
)
print(df[["revenue", "revenue_level"]].head(10))
```

```
...      revenue revenue_level
0   548.9715   High Revenue
1    80.2200    Low Revenue
2   340.5255   High Revenue
3   489.0480   High Revenue
4   634.3785   High Revenue
5   627.6165   High Revenue
6   433.6920   High Revenue
7   772.3800   High Revenue
8    76.1460    Low Revenue
9   172.7460    Low Revenue
```

#Q9. Create the revenue_level column

```
df["revenue_level"] = np.where(
    df["revenue"] >= median_revenue,
    "High Revenue",
    "Low Revenue"
)
print(df[["revenue", "revenue_level"]].head(10))
```

```
      revenue revenue_level
0   548.9715   High Revenue
1    80.2200    Low Revenue
2   340.5255   High Revenue
3   489.0480   High Revenue
4   634.3785   High Revenue
5   627.6165   High Revenue
6   433.6920   High Revenue
7   772.3800   High Revenue
8    76.1460    Low Revenue
9   172.7460    Low Revenue
```

#Q10. Calculate number of High Revenue and Low Revenue transactions for each product line

```
revenue_counts = pd.crosstab(
    df["product_line"],
    df["revenue_level"]
)
print(revenue_counts)
```

```
revenue_level      High Revenue  Low Revenue
product_line
Electronic accessories          79          91
Fashion accessories            84          94
Food and beverages             86          88
Health and beauty              81          71
Home and lifestyle             82          78
Sports and travel              88          78
```

#Q11. Calculate probability of High Revenue and Low Revenue

```
probability = pd.crosstab(
    df["product_line"],
    df["revenue_level"],
```

```

normalize="index"
)
print(probability)

```

```

***  revenue_level      High Revenue  Low Revenue
    product_line
Electronic accessories      0.464706      0.535294
Fashion accessories        0.471910      0.528090
Food and beverages         0.494253      0.505747
Health and beauty          0.532895      0.467105
Home and lifestyle         0.512500      0.487500
Sports and travel          0.530120      0.469880

```

#Q12. Verify probabilities add up to 1

```

probability_sum = probability.sum(axis=1)
print(probability_sum)

```

```

***  product_line
    Electronic accessories      1.0
    Fashion accessories        1.0
    Food and beverages         1.0
    Health and beauty          1.0
    Home and lifestyle         1.0
    Sports and travel          1.0
    dtype: float64

```

#Q13. Calculate average revenue for High and Low Revenue transactions

```

payoff = df.groupby(
    ["product_line", "revenue_level"]
)["revenue"].mean()
print(payoff)

```

```

***  product_line      revenue_level
    Electronic accessories  High Revenue      537.006152
                             Low Revenue      130.923577
    Fashion accessories    High Revenue      512.329375
                             Low Revenue      119.896037
    Food and beverages     High Revenue      515.127680
                             Low Revenue      134.589358
    Health and beauty      High Revenue      497.221407
                             Low Revenue      125.616972
    Home and lifestyle     High Revenue      529.582738
                             Low Revenue      133.796519
    Sports and travel      High Revenue      518.546438
                             Low Revenue      121.676154
    Name: revenue, dtype: float64

```

#Q14. Create DataFrame containing Product Line, Revenue Level, Probability and Payoff

```

counts = df.groupby(
    ["product_line", "revenue_level"]
).size().reset_index(name="Count")
total_transactions = df.groupby(
    "product_line"
).size().reset_index(name="Total")
decision_df = counts.merge(

```

```

total_transactions,
on="product_line"
)
decision_df["Probability"] = (
    decision_df["Count"] / decision_df["Total"]
)
decision_df["Payoff"] = df.groupby(
    ["product_line", "revenue_level"]
)["revenue"].mean().values
decision_df = decision_df[
    ["product_line", "revenue_level", "Probability", "Payoff"]
]
print(decision_df)

```

...	product_line	revenue_level	Probability	Payoff
0	Electronic accessories	High Revenue	0.464706	537.006152
1	Electronic accessories	Low Revenue	0.535294	130.923577
2	Fashion accessories	High Revenue	0.471910	512.329375
3	Fashion accessories	Low Revenue	0.528090	119.896037
4	Food and beverages	High Revenue	0.494253	515.127680
5	Food and beverages	Low Revenue	0.505747	134.589358
6	Health and beauty	High Revenue	0.532895	497.221407
7	Health and beauty	Low Revenue	0.467105	125.616972
8	Home and lifestyle	High Revenue	0.512500	529.582738
9	Home and lifestyle	Low Revenue	0.487500	133.796519
10	Sports and travel	High Revenue	0.530120	518.546438
11	Sports and travel	Low Revenue	0.469880	121.676154

Q15. Identify decision alternatives and uncertain outcomes Decision Alternatives

The six product lines are the decision alternatives:

Electronic accessories, Fashion accessories, Food and beverages, Health and beauty, Home and lifestyle, Sports and travel, Uncertain Outcomes.

There are two possible revenue outcomes:

High Revenue, Low Revenue.

This follows directly from the practical's requirement that each product line be a decision alternative and High/Low Revenue be uncertain outcomes.

#Q16. Construct the decision tree

for product in df["product_line"].unique():

```

print("\n" + product)
print(" |")
print(" +-- High Revenue")
print(" |   Probability:",
    round(decision_df[
        (decision_df["product_line"] == product) &
        (decision_df["revenue_level"] == "High Revenue")
    ]["Probability"].iloc[0], 4))
print(" |   Payoff:",
    round(decision_df[
        (decision_df["product_line"] == product) &
        (decision_df["revenue_level"] == "High Revenue")
    ]["Payoff"].iloc[0], 4))

```

```
][ "Payoff"].iloc[0], 2))
```

```
print(" |")
print(" +-- Low Revenue")
print("      Probability:",
      round(decision_df[
        (decision_df["product_line"] == product) &
        (decision_df["revenue_level"] == "Low Revenue")
      ][ "Probability"].iloc[0], 4))
print("      Payoff:",
      round(decision_df[
        (decision_df["product_line"] == product) &
        (decision_df["revenue_level"] == "Low Revenue")
      ][ "Payoff"].iloc[0], 2))
```

Health and beauty

```
|
+-- High Revenue
|      Probability: 0.5329
|      Payoff: 497.22
|
+-- Low Revenue
      Probability: 0.4671
      Payoff: 125.62
```

Electronic accessories

```
|
+-- High Revenue
|      Probability: 0.4647
|      Payoff: 537.01
|
+-- Low Revenue
      Probability: 0.5353
      Payoff: 130.92
```

Home and lifestyle

```
|
+-- High Revenue
|      Probability: 0.5125
|      Payoff: 529.58
|
+-- Low Revenue
      Probability: 0.4875
```

Sports and travel

```
...
|
+-- High Revenue
|      Probability: 0.5301
|      Payoff: 518.55
|
+-- Low Revenue
      Probability: 0.4699
      Payoff: 121.68
```

Food and beverages

```
|
+-- High Revenue
|      Probability: 0.4943
|      Payoff: 515.13
|
+-- Low Revenue
      Probability: 0.5057
      Payoff: 134.59
```

Fashion accessories

```
|
+-- High Revenue
|      Probability: 0.4719
|      Payoff: 512.33
|
+-- Low Revenue
```

#Q17. Display probability and payoff at every outcome

```
for product in decision_df["product_line"].unique():
```

```
    print("\nProduct Line:", product)
```

```
    product_data = decision_df[
```

```
        decision_df["product_line"] == product
```

```
    ]
```

```
    for _, row in product_data.iterrows():
```

```
        print(
```

```
            row["revenue_level"],
```

```
            "-> Probability:",
```

```
            round(row["Probability"], 4),
```

```
            ", Payoff:",
```

```
            round(row["Payoff"], 2)
```

```
        )
```

```
...
```

```
Product Line: Electronic accessories
```

```
High Revenue -> Probability: 0.4647 , Payoff: 537.01
```

```
Low Revenue -> Probability: 0.5353 , Payoff: 130.92
```

```
Product Line: Fashion accessories
```

```
High Revenue -> Probability: 0.4719 , Payoff: 512.33
```

```
Low Revenue -> Probability: 0.5281 , Payoff: 119.9
```

```
Product Line: Food and beverages
```

```
High Revenue -> Probability: 0.4943 , Payoff: 515.13
```

```
Low Revenue -> Probability: 0.5057 , Payoff: 134.59
```

```
Product Line: Health and beauty
```

```
High Revenue -> Probability: 0.5329 , Payoff: 497.22
```

```
Low Revenue -> Probability: 0.4671 , Payoff: 125.62
```

```
Product Line: Home and lifestyle
```

```
High Revenue -> Probability: 0.5125 , Payoff: 529.58
```

```
Low Revenue -> Probability: 0.4875 , Payoff: 133.8
```

```
Product Line: Sports and travel
```

```
High Revenue -> Probability: 0.5301 , Payoff: 518.55
```

```
Low Revenue -> Probability: 0.4699 , Payoff: 121.68
```

#Q18. Visualization of total revenue for each product line

```
total_revenue = df.groupby("product_line")["revenue"].sum()
```

```
plt.figure(figsize=(10, 5))
```

```
total_revenue.plot(kind="bar")
```

```
plt.title("Total Revenue by Product Line")
```

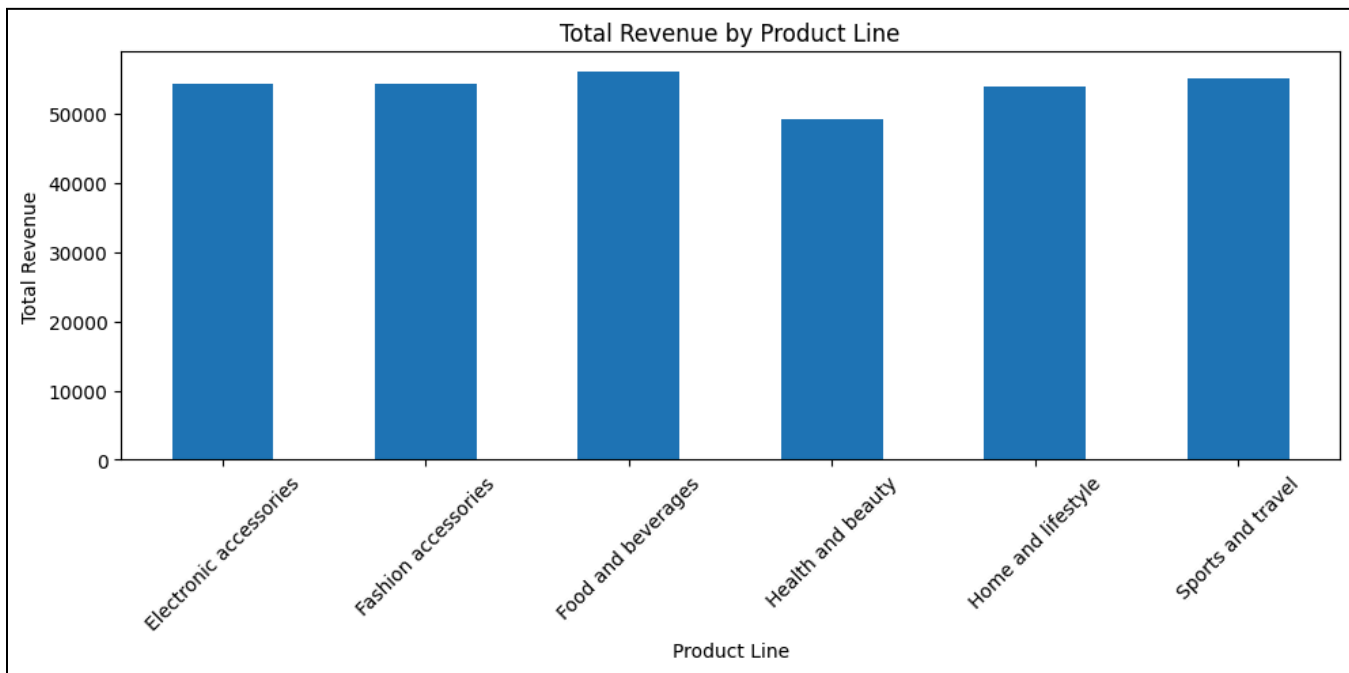
```
plt.xlabel("Product Line")
```

```
plt.ylabel("Total Revenue")
```

```
plt.xticks(rotation=45)
```

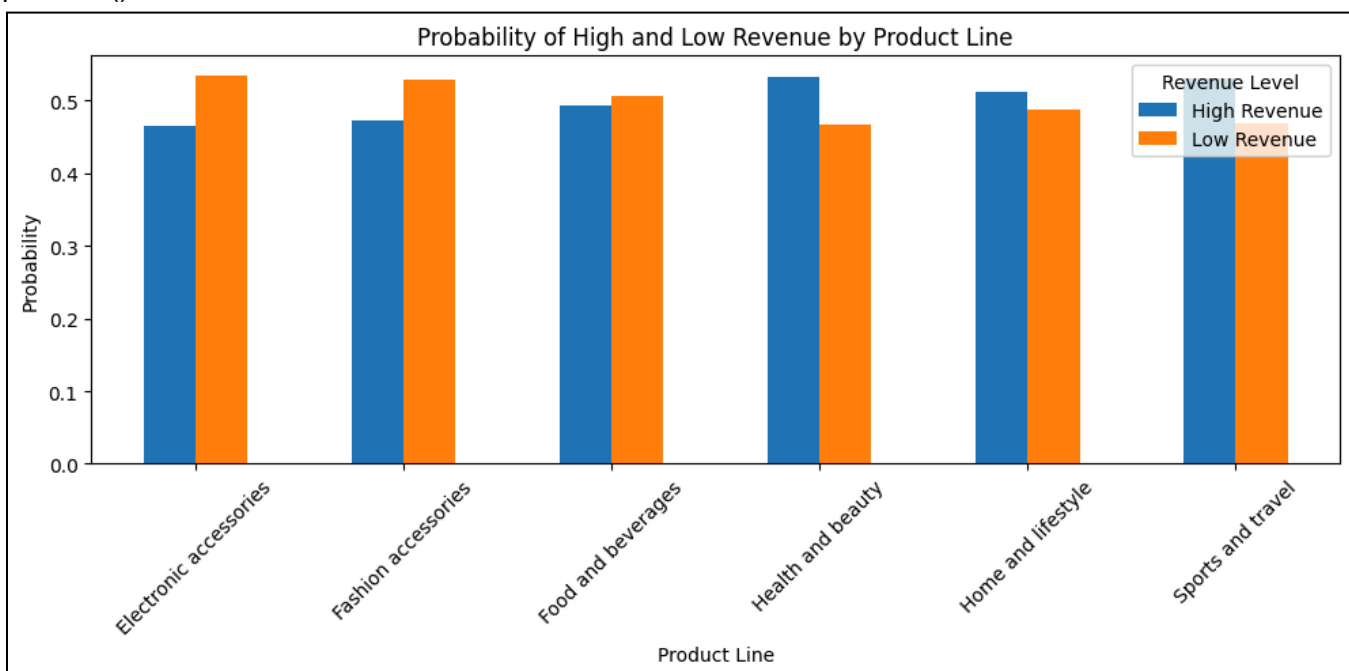
```
plt.tight_layout()
```

```
plt.show()
```



#Q19. Visualization of High/Low Revenue probabilities

```
probability.plot(  
    kind="bar",  
    figsize=(10, 5)  
)  
plt.title("Probability of High and Low Revenue by Product Line")  
plt.xlabel("Product Line")  
plt.ylabel("Probability")  
plt.xticks(rotation=45)  
plt.legend(title="Revenue Level")  
plt.tight_layout()  
plt.show()
```




```
#Q20. Product line with highest probability of High Revenue
high_probability = probability["High Revenue"]
highest_high_probability = high_probability.idxmax()
highest_probability_value = high_probability.max()
print("Product line:", highest_high_probability)
print("Probability:", highest_probability_value)
```

```
Product line: Health and beauty
Probability: 0.5328947368421053
```

```
# Q21-Q23: Calculate Expected Value for Home and lifestyle
# Select Home and lifestyle
home_data = decision_df[
    decision_df["product_line"] == "Home and lifestyle"
]
# Get probability and payoff for High Revenue
high = home_data[
    home_data["revenue_level"] == "High Revenue"
].iloc[0]
# Get probability and payoff for Low Revenue
low = home_data[
    home_data["revenue_level"] == "Low Revenue"
].iloc[0]
# Calculate Expected Value
ev = (
    high["Probability"] * high["Payoff"]
    + low["Probability"] * low["Payoff"]
)
print("High Revenue Probability:", round(high["Probability"], 4))
print("High Revenue Payoff:", round(high["Payoff"], 4))
print("Low Revenue Probability:", round(low["Probability"], 4))
print("Low Revenue Payoff:", round(low["Payoff"], 4))
print("\nExpected Value:", round(ev, 4))
```

```
High Revenue Probability: 0.5125
High Revenue Payoff: 529.5827
Low Revenue Probability: 0.4875
Low Revenue Payoff: 133.7965

Expected Value: 336.637
```

```
#Q24. Calculate Expected Value for every product line
decision_df["EV_Component"] = (
    decision_df["Probability"] *
    decision_df["Payoff"]
)

expected_value = decision_df.groupby(
    "product_line"
)["EV_Component"].sum()

print(expected_value)
```

```

product_line
Electronic accessories    319.632538
Fashion accessories       305.089298
Food and beverages       322.671517
Health and beauty        323.643020
Home and lifestyle       336.636956
Sports and travel        332.065220
Name: EV_Component, dtype: float64

```

#Q25. Create the required Expected Value DataFrame

```

ev_df = decision_df.pivot(
    index="product_line",
    columns="revenue_level",
    values=["Probability", "Payoff"]
)
ev_final = pd.DataFrame({
    "Product Line": ev_df.index,
    "High Revenue Probability":
        ev_df[("Probability", "High Revenue")].values,
    "High Revenue Payoff":
        ev_df[("Payoff", "High Revenue")].values,
    "Low Revenue Probability":
        ev_df[("Probability", "Low Revenue")].values,
    "Low Revenue Payoff":
        ev_df[("Payoff", "Low Revenue")].values
})
ev_final["Expected Value"] = (
    ev_final["High Revenue Probability"] *
    ev_final["High Revenue Payoff"]
    +
    ev_final["Low Revenue Probability"] *
    ev_final["Low Revenue Payoff"]
)
print(ev_final)

```

```

...      Product Line  High Revenue Probability  High Revenue Payoff  \
0  Electronic accessories      0.464706      537.006152
1   Fashion accessories      0.471910      512.329375
2   Food and beverages      0.494253      515.127680
3   Health and beauty      0.532895      497.221407
4   Home and lifestyle      0.512500      529.582738
5   Sports and travel      0.530120      518.546438

      Low Revenue Probability  Low Revenue Payoff  Expected Value
0      0.535294      130.923577      319.632538
1      0.528090      119.896037      305.089298
2      0.505747      134.589358      322.671517
3      0.467105      125.616972      323.643020
4      0.487500      133.796519      336.636956
5      0.469880      121.676154      332.065220

```

#Q26. Compare Expected Values

```
print(
    ev_final[
        ["Product Line", "Expected Value"]
    ].sort_values(
        "Expected Value",
        ascending=False
    )
)
```

```
...
      Product Line  Expected Value
4    Home and lifestyle    336.636956
5    Sports and travel    332.065220
3    Health and beauty    323.643020
2    Food and beverages    322.671517
0  Electronic accessories    319.632538
1    Fashion accessories    305.089298
```

#Q27. Product line with highest Expected Value

```
highest_ev = ev_final.loc[
    ev_final["Expected Value"].idxmax()
]
print(highest_ev)
```

```
Product Line      Home and lifestyle
High Revenue Probability    0.5125
High Revenue Payoff    529.582738
Low Revenue Probability    0.4875
Low Revenue Payoff    133.796519
Expected Value    336.636956
Name: 4, dtype: object
```

#Q28. Product line with lowest Expected Value

```
lowest_ev = ev_final.loc[
    ev_final["Expected Value"].idxmin()
]
print(lowest_ev)
```

```
Product Line      Fashion accessories
High Revenue Probability    0.47191
High Revenue Payoff    512.329375
Low Revenue Probability    0.52809
Low Revenue Payoff    119.896037
Expected Value    305.089298
Name: 1, dtype: object
```

#Q29. Rank product lines from highest to lowest Expected Value

```
ranking = ev_final[
    ["Product Line", "Expected Value"]
].sort_values(
    "Expected Value",
    ascending=False
)
```

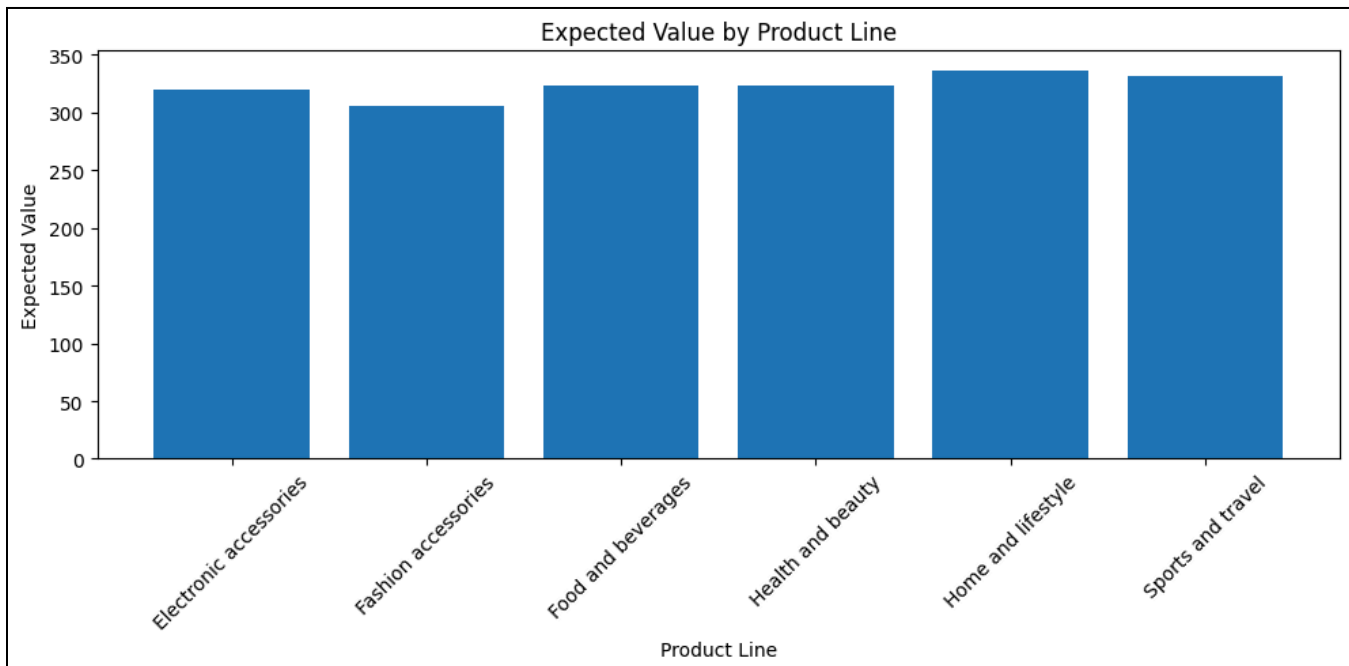
```
print(ranking)
```

	Product Line	Expected Value
4	Home and lifestyle	336.636956
5	Sports and travel	332.065220
3	Health and beauty	323.643020
2	Food and beverages	322.671517
0	Electronic accessories	319.632538
1	Fashion accessories	305.089298

```
#Q30. Bar chart comparing Expected Values
```

```
plt.figure(figsize=(10, 5))
```

```
plt.bar(
    ev_final["Product Line"],
    ev_final["Expected Value"]
)
plt.title("Expected Value by Product Line")
plt.xlabel("Product Line")
plt.ylabel("Expected Value")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
# Q31. Compare highest High Revenue Probability
```

```
# with highest Expected Value
```

```
# Find product line with highest High Revenue probability
```

```
highest_probability = ev_final.loc[
    ev_final["High Revenue Probability"].idxmax()
]
```

```
# Find product line with highest Expected Value
```

```
highest_ev = ev_final.loc[
    ev_final["Expected Value"].idxmax()
]
```

```
print("Product line with highest High Revenue Probability:")
print(highest_probability["Product Line"])
print("Probability:",
      round(highest_probability["High Revenue Probability"], 4))
print("\nProduct line with highest Expected Value:")
print(highest_ev["Product Line"])
print("Expected Value:",
      round(highest_ev["Expected Value"], 2))
print("\nComparison:")
if highest_probability["Product Line"] == highest_ev["Product Line"]:
    print("Both are the same product line.")
else:
    print("They are different product lines.")
```

```
... Product line with highest High Revenue Probability:
Health and beauty
Probability: 0.5329

Product line with highest Expected Value:
Home and lifestyle
Expected Value: 336.64

Comparison:
They are different product lines.
```

Q32. Check whether both product lines are the same

```
if highest_probability["Product Line"] == highest_ev["Product Line"]:
    print("Yes, they are the same product line.")
else:
    print("No, they are not the same product line.")
```

```
... No, they are not the same product line.
```

#Q33. Change probability of one outcome and recalculate Expected Value

```
high_probability = 0.70
low_probability = 0.30
high_payoff = 512.329375
low_payoff = 119.896037
new_ev = (
    high_probability * high_payoff
    +
    low_probability * low_payoff
)
print("New Expected Value:", new_ev)
```

```
... New Expected Value: 394.5993736
```

#Q34. Change payoff of one outcome and recalculate Expected Value

```
prob_high = 0.471910
prob_low = 0.528090
new_high_payoff = 600
low_payoff = 119.896037
```

```
new_ev = (  
    prob_high * new_high_payoff  
    +  
    prob_low * low_payoff  
)  
print("New Expected Value:", new_ev)
```

```
... New Expected Value: 346.46189817933003
```

#Q35. Python program to automatically calculate Expected Value and select the best product line

```
import pandas as pd  
import numpy as np  
# Load dataset  
df = pd.read_csv("supermarket_sales - supermarket_sales.csv")  
# Calculate median revenue  
median_revenue = df["revenue"].median()  
# Classify revenue  
df["revenue_level"] = np.where(  
    df["revenue"] >= median_revenue,  
    "High Revenue",  
    "Low Revenue"  
)  
# Calculate count  
counts = df.groupby(  
    ["product_line", "revenue_level"]  
) .size().reset_index(name="Count")  
# Calculate total transactions for each product line  
totals = df.groupby(  
    "product_line"  
) .size().reset_index(name="Total")  
# Merge  
result = counts.merge(  
    totals,  
    on="product_line"  
)  
# Calculate probability  
result["Probability"] = (  
    result["Count"] / result["Total"]  
)  
# Calculate payoff  
payoffs = df.groupby(  
    ["product_line", "revenue_level"]  
) ["revenue"].mean().reset_index(  
    name="Payoff"  
)  
  
result = result.merge(  
    payoffs,  
    on=["product_line", "revenue_level"]  
)  
# Calculate EV component
```

```

result["EV_Component"] = (
    result["Probability"] *
    result["Payoff"]
)
# Calculate Expected Value
ev = result.groupby(
    "product_line"
)["EV_Component"].sum().reset_index()
ev.columns = [
    "Product Line",
    "Expected Value"
]
# Sort from highest to lowest
ev = ev.sort_values(
    "Expected Value",
    ascending=False
)
print("Expected Value of each product line:")
print(ev)
# Select highest Expected Value
best_product = ev.iloc[0]
print("\nBest Product Line:")
print(best_product["Product Line"])
print("Highest Expected Value:",
      round(best_product["Expected Value"], 2))

```

```

Expected Value of each product line:
      Product Line  Expected Value
4  Home and lifestyle    336.636956
5  Sports and travel    332.065220
3  Health and beauty    323.643020
2  Food and beverages    322.671517
0  Electronic accessories  319.632538
1  Fashion accessories    305.089298

Best Product Line:
Home and lifestyle
Highest Expected Value: 336.64

```

```

# Q36. Display final decision tree with Expected Value
print("FINAL DECISION TREE")
print("=====")
for _, row in ev_final.iterrows():
    print("\nProduct Line:", row["Product Line"])
    print("Expected Value:", round(row["Expected Value"], 2))
    print(" |")
    print(" +-- High Revenue")
    print(" |   Probability:",
          round(row["High Revenue Probability"], 4))
    print(" |   Payoff:",
          round(row["High Revenue Payoff"], 2))
    print(" |")

```

```
print(" +-- Low Revenue")
print("      Probability:",
      round(row["Low Revenue Probability"], 4))
print("      Payoff:",
      round(row["Low Revenue Payoff"], 2))
```

FINAL DECISION TREE

=====

Product Line: Electronic accessories
Expected Value: 319.63

```
|
+-- High Revenue
|   Probability: 0.4647
|   Payoff: 537.01
|
+-- Low Revenue
|   Probability: 0.5353
|   Payoff: 130.92
```

Product Line: Fashion accessories
Expected Value: 305.09

```
|
+-- High Revenue
|   Probability: 0.4719
|   Payoff: 512.33
|
+-- Low Revenue
|   Probability: 0.5281
|   Payoff: 119.9
```

Product Line: Food and beverages
Expected Value: 322.67

```
|
+-- High Revenue
|   Probability: 0.4943
|   Payoff: 515.13
|
+-- Low Revenue
|   Probability: 0.5057
|   Payoff: 134.59
```

```
*** Product Line: Food and beverages
Expected Value: 322.67
```

```
|
+-- High Revenue
|   Probability: 0.4943
|   Payoff: 515.13
|
+-- Low Revenue
|   Probability: 0.5057
|   Payoff: 134.59
```

Product Line: Health and beauty
Expected Value: 323.64

```
|
+-- High Revenue
|   Probability: 0.5329
|   Payoff: 497.22
|
+-- Low Revenue
|   Probability: 0.4671
|   Payoff: 125.62
```

Product Line: Home and lifestyle
Expected Value: 336.64

```
|
+-- High Revenue
|   Probability: 0.5125
|   Payoff: 529.58
|
+-- Low Revenue
|   Probability: 0.4875
|   Payoff: 133.8
```

Q37. Business Recommendation

Recommendation

Based on the Expected Value analysis, the business should select:

Home and lifestyle

It has the highest Expected Value of 336.64 among all six product lines.

Although Health and beauty has the highest probability of High Revenue at 53.29%, it does not have the highest Expected Value.

Home and lifestyle has:

High Revenue Probability = 51.25% High Revenue Payoff = 529.58

Low Revenue Probability = 48.75% Low Revenue Payoff = 133.80

Expected Value = 336.64

Therefore, according to the Expected Value decision criterion:

Home and lifestyle should be selected because it provides the highest expected revenue among all available product-line alternatives.

